

Communication & Event-Driven Architecture

Zephyr RTOS Intermediate Course





Quick Recap — Lecture 3



- Timers: `k_timer`, one-shot vs periodic, expiry context
- Workqueues: system vs custom, serialisation, `k_work_delayable`
- Deferred execution patterns: bottom-half, debounce, periodic
- `k_poll`: waiting on multiple event sources
- Reducing unnecessary threads



Today's Agenda



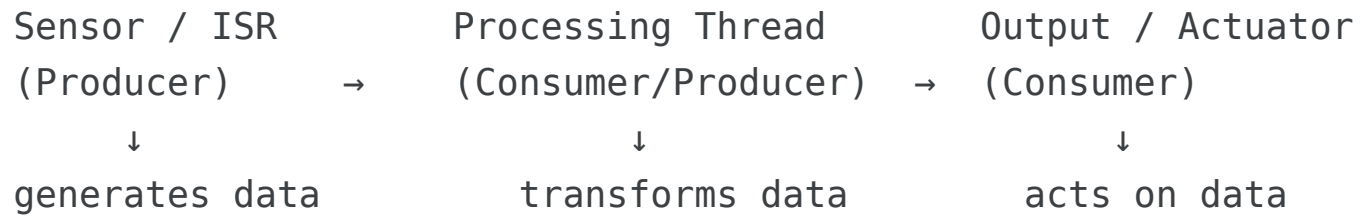
- Producer-consumer model: data flow and ownership
- Message queues (`k_msgq`): buffered copy-based IPC
- FIFOs & Pipes: pointer passing and byte streams
- Backpressure & overflow strategies
- Event-driven architecture & publish-subscribe
- Zbus: channels, observers, listeners vs subscribers
- Module boundaries and coupling



Producer-Consumer Model



Data Flow in Embedded Systems



- **Ownership:** who is responsible for the data at each stage?
- **Lifetime:** how long does data need to exist?
- **Timing:** can the consumer keep up with the producer?



Two Transfer Strategies



Strategy	How	When
Copy	Data is duplicated into the primitive's buffer	Small messages, ISR safety needed, no shared lifetime required
Reference	A pointer to data is transferred	Large data, zero-copy needed, producer controls lifetime

- **Copy:** safe, simple, costs memory bandwidth
- **Reference:** fast, zero-copy, requires careful lifetime management



Message Queues



k_msgq

Fundamentals



A message queue is a FIFO ring buffer that **copies** fixed-size messages between producers and consumers

```
K_MSGQ_DEFINE(my_msgq, sizeof(struct sensor_msg), max_msgs, align);
```

Key APIs:

- `k_msgq_put(&my_msgq, &msg, K_FOREVER)` send, waits if full
- `k_msgq_get(&my_msgq, &msg, K_FOREVER)` receive, waits if empty
- `k_msgq_peek(&my_msgq, &msg)` look without removing
- `k_msgq_purge(&my_msgq)` discard all pending messages
- `k_msgq_num_used_get(&my_msgq)` check fill level



Message Queue Waiting Behaviour



```
// Producer
k_msgq_put(&my_msgq, &msg, K_FOREVER); // waiting indefinitely if full
k_msgq_put(&my_msgq, &msg, K_NO_WAIT); // return -ENOMSG if queue full with K_NO_WAIT
k_msgq_put(&my_msgq, &msg, K_MSEC(50)); // return -EAGAIN after timeout

// Consumer
k_msgq_get(&my_msgq, &msg, K_FOREVER); // wait until a message arrives
k_msgq_get(&my_msgq, &msg, K_NO_WAIT); // return -ENOMSG if queue empty with K_NO_WAIT

// Urgent messages – sent to the front of the queue
k_msgq_put_front(&my_msgq, &urgent_msg, K_NO_WAIT);
```

- When producer blocks it enters **Waiting** state, no CPU wasted
- When the queue has space again highest-priority waiting producer unblocks



When to Use

`k_msgq`



● Use `k_msgq` **when**:

- Messages are small, fixed-size structs
- ISR-to-thread or thread-to-thread communication
- You want safe copy semantics (no shared-lifetime issues)
- Multiple producers or multiple consumers are possible

● Avoid `k_msgq` **when**:

- Messages are variable-length (use pipe or FIFO + allocation)
- Data is large and memcpy cost is unacceptable (use FIFO + pointer)
- You need broadcast to multiple consumers (use zbus)



FIFOs & Pipes



k_fifo

Pointer Queue



A FIFO transfers **pointers**, not copies. Ownership moves from producer to consumer.

```
K_FIFO_DEFINE(my_fifo);

struct packet {
    void *fifo_reserved;    // required FIRST field – kernel uses it for linking
    uint8_t data[64];
    size_t len;
};

// Producer
struct packet *pkt = alloc_packet();
k_fifo_put(&my_fifo, pkt);    // pointer transferred, do not touch pkt after this

// Consumer
struct packet *pkt = k_fifo_get(&my_fifo, K_FOREVER);
process(pkt);
free_packet(pkt);            // consumer owns and frees
```

`k_fifo`

Lifetime Rules



Because FIFO transfers pointers, **data must outlive the queue item**

Memory type	Lifetime	Safe for FIFO?
Stack variable	Until function returns	Never
Heap (<code>k_malloc</code>)	Until <code>k_free</code>	If freed by consumer
<code>k_mem_slab</code> block	Until <code>k_mem_slab_free</code>	Preferred for RT
Static / global	Entire program lifetime	If ownership is clear

● The classic bug. Never do this:

```
struct packet pkt;           // stack object, frame destroyed on function return
k_fifo_put(&my_fifo, &pkt); // consumer gets a dangling pointer
```



Real-time pattern. Slab-backed FIFO:



```
/* One slab: fixed-size blocks, O(1) alloc/free, no fragmentation */
K_MEM_SLAB_DEFINE(pkt_slab, sizeof(struct packet), 16, 4);

// Producer: alloc from slab, fill, hand off
struct packet *pkt;
if (k_mem_slab_alloc(&pkt_slab, (void **)&pkt, K_NO_WAIT) == 0) {
    fill(pkt);
    k_fifo_put(&my_fifo, pkt);    // ownership transfers here
}

// Consumer: process, return block to slab
pkt = k_fifo_get(&my_fifo, K_FOREVER);
process(pkt);
k_mem_slab_free(&pkt_slab, pkt);
```



k_pipe

Byte Stream



A pipe transfers a continuous stream of bytes

```
K_PIPE_DEFINE(my_pipe, 256, 4);

// Writer
k_pipe_write(&my_pipe, tx_buf, len, K_FOREVER);

// Reader
size_t n = k_pipe_read(&my_pipe, rx_buf, sizeof(rx_buf), K_NO_WAIT);
// n bytes were read – may be less than requested; loop or frame at application level
```

- Partial reads and writes are normal, **framing is your responsibility**
- ISR-safe only with `K_NO_WAIT`, waiting is forbidden in ISR context
- `k_pipe_reset()` discards all buffered data, it is useful for protocol error recovery



Choosing the Right Primitive



Primitive	Best For
<code>k_msgq</code>	Commands, fixed-size events, ISR-to-thread
<code>k_fifo</code>	Large buffers, variable-size packets
<code>k_pipe</code>	UART streams, audio, protocol framing



DEMO 1: Message Queue Pipeline



Backpressure & Overflow



The Slow Consumer Problem



Producer (fast) → [**Queue**] → Consumer (slow)
50 ms/msg capacity = 10 200 ms/msg

Strategy	Behaviour	Use case
Block	Producer waits until consumer catches up	Data must not be lost
Drop newest	Discard incoming message if queue is full	Older data is still valid
Drop oldest	<code>k_msgq_purge()</code> then retry	Latest data is more important



Zbus



Zbus Concepts



Publisher
↓
Listener (sync) ← Channel → Subscriber (async)

- **Channel:** shared typed message + observer list
- **Listener:** synchronous callback in publisher context
- **Subscriber:** asynchronous processing in its own thread



Defining Zbus Channel



```
// sensor_iface.h – owned by nobody, shared by all
struct sensor_data {
    int32_t  temperature_mc;
    uint32_t timestamp_ms;
};

ZBUS_CHAN_DEFINE(
    sensor_chan,
    struct sensor_data,
    NULL, NULL, // validator (optional) and user data
    ZBUS_OBSERVERS(display_lis, logger_sub),
    ZBUS_MSG_INIT(.temperature_mc = 0)
);
```

- The channel header **is** the interface – changing the struct breaks all users at compile time
- Producer and all consumers depend only on this header, not on each other



```
// Publisher – any thread or ISR
struct sensor_data data = { .temperature_mc = 24000 };
zbus_chan_pub(&sensor_chan, &data, K_MSEC(100));
```

```
// Listener – synchronous, runs in publisher's thread
void display_cb(const struct zbus_channel *chan)
{
    const struct sensor_data *msg = zbus_chan_const_msg(chan);
    display_update(msg->temperature_mc); // fast update only – no blocking
}
ZBUS_LISTENER_DEFINE(display_lis, display_cb);
```

- `zbus_chan_const_msg()` pointer is valid **only inside the callback** – do not store it
- Listener execution time directly increases publish latency for all observers



```
ZBUS_SUBSCRIBER_DEFINE(logger_sub, 4); // internal notification queue depth

void logger_thread(void)
{
    const struct zbus_channel *chan;
    while (!zbus_sub_wait(&logger_sub, &chan, K_FOREVER)) {
        struct sensor_data msg;
        zbus_chan_read(chan, &msg, K_MSEC(100));
        LOG_INF("Temp: %d mC", msg.temperature_mc);
    }
}
```

- `zbus_sub_wait` blocks until any observed channel publishes
- `zbus_chan_read` explicitly copies the latest value out. It may differ from what triggered the notification if the channel was updated again



Listener vs Subscriber



	Listener	Subscriber
Execution context	Publisher's thread	Observer's own thread
Notification guarantee	Every publish	May miss if queue full
Can block?	Never	Yes
Latency	Lowest	Scheduling-dependent
Use case	Flag set, LED toggle, immediate reaction	Logging, network send, slow I/O

- **When in doubt:** use a subscriber, it is safer and more decoupled
- **Common pattern:** listener sets an atomic flag, the dedicated thread does the heavy work



DEMO 2: Zbus Pub-Sub



Module Boundaries & Coupling



What Is Coupling?



Tight coupling (k_msgq direct)

sensor.c knows about display.c
sensor.c knows about logger.c
sensor.c knows about alarm.c

Add new observer? Edit sensor.c

Loose coupling (zbus)

sensor.c knows only sensor_chan
display.c knows only sensor_chan
logger.c knows only sensor_chan

Add new observer? Edit only the new module

- **Tight coupling:** fast, simple, breaks when you scale
- **Loose coupling:** more infrastructure, survives requirements changes



Who defines the channel?

Option A: Producer owns

`sensor.c` defines `sensor_chan`
All observers depend on `sensor.c`

→ Circular dependency risk

Option B: Shared contract

`sensor_iface.h` defines `sensor_chan`
`sensor.c` and all observers depend on `sensor_iface.h`

→ Clean dependency graph

Rule: channels and message structs belong in a dedicated interface header, not in any module's source file



Selection Guide



Situation	Tool	Why
ISR-to-thread, fixed struct	<code>k_msgq</code>	Copy-safe, ISR-compatible
Large data, pointer transfer	<code>k_fifo</code>	Zero-copy, any size
Byte stream (UART, audio)	<code>k_pipe</code>	Streaming, partial R/W
One event, multiple consumers	<code>zbus</code> channel	Broadcast, decoupled
Fast reaction needed	<code>zbus</code> listener	Synchronous callback
Slow/blocking processing	<code>zbus</code> subscriber	Own thread, can block
Multiple sources, one waiter	<code>k_poll</code>	Covered in Lecture 3



What Can Go Wrong



- **Subscribing but never reading:** FIFO fills, notifications silently lost
- **Listener that blocks:** publisher hangs, priority inversion possible
- **Pointer in msgq:** dangling pointer if producer frees before consumer reads
- **No alignment on K_MSGQ_DEFINE:** misaligned access, hard fault on ARM
- **Channel mutex timeout ignored:** missed publications, stale data consumed
- **Mixing subscriber and claim/finish:** deadlock if channel is claimed and never finished



Homework



- **Goal:** Build a small event-driven system with zbus
- **Tasks:**
 - Create one zbus channel for sensor data
 - Publish simulated sensor samples every 100 ms
 - Add one listener for fast display updates
 - Add one subscriber for slower logging
- Push tag `l4-task1`

Thank You





Resources

1. [Message Queues](#)
2. [FIFOs](#)
3. [Pipes](#)
4. [Zbus](#)

